Guia completo de instalação e uso do Vladmir

Escopo: versão do código-fonte no branch main, consultada em 14/10/2026. Os passos abaixo não
afirmam que o software foi instalado ou executado neste ambiente.

1. O que é o Vladmir

Vladmir é um assistente local para Linux com aplicativo desktop, modelos servidos pelo Ollama, memória e agenda em arquivos locais e integrações opcionais. Há código separado para Android, Companion e Wear OS. Muitas funções dependem de programas externos, permissões, modelos e configuração. A instalação não inclui pesos de IA, credenciais, APK pronto nem servidor remoto configurado.

2. Requisitos básicos

* Linux e Python 3.11 ou superior.
* Ollama instalado e iniciado em http://127.0.0.1:11434.
* Um modelo de chat e, se desejado, um modelo de programação já baixados no Ollama.
* Para abrir a interface desktop, PyQt6 e as dependências do requirements.txt.
* Espaço para o ambiente Python, modelos e dados. O espaço ocupado pelos pesos depende dos modelos escolhidos.

Recursos opcionais têm dependências próprias: ffmpeg e PulseAudio/PipeWire para áudio e câmera; Whisper e arquivos de modelo para transcrição; Piper e voz instalada para síntese; ClamAV para varredura; gh para a autenticação GitHub; Android Studio/SDK/JDK para compilar os apps; e servidor MCP local habilitado dentro da IDE JetBrains.

3. Instalar o desktop

O procedimento documentado no README instala o checkout e um ambiente virtual em sua pasta de **usuário:**

```
bash
mkdir -p ~/.local/share/vladmir
git clone https://github.com/thu12dta3/Vladmir.git ~/.local/share/vladmir/source
cd ~/.local/share/vladmir/source
python -m venv ~/.local/share/vladmir/core-env
~/.local/share/vladmir/core-env/bin/pip install -r requirements.txt
install -Dm755 scripts/vladmir ~/.local/bin/vladmir
install -Dm755 scripts/vladmir-desktop ~/.local/bin/vladmir-desktop
```

O README também contém comandos para instalar ícone e atalho do menu de aplicativos. Se ~/.local/bin não estiver no PATH, abra o aplicativo pelo caminho completo ou ajuste o PATH na sessão do usuário.

Antes de iniciar, consulte as verificações e opções do projeto:

```
bash
cd ~/.local/share/vladmir/source
./scripts/vladmir --check
```

Esse check consulta dependências, configurações e disponibilidade de modelos. Ele não prova que uma câmera desconectada funciona, que o áudio sai no dispositivo correto ou que um telefone pareado responde.

4. Configuração e modelos

Crie ~/.config/vladmir/config.json a partir de config/config.example.json e proteja o arquivo:

bash
mkdir -p ~/.config/vladmir
cp config/config.example.json ~/.config/vladmir/config.json
chmod 600 ~/.config/vladmir/config.json

Edite o JSON para escolher identificadores que existam no Ollama. A configuração de exemplo usa qwen3-vl:4b para conversa e qwen2.5-coder:7b para programação, mas a máquina precisa ter esses modelos instalados. Vladmir não baixa modelos sozinho. Confira os modelos locais com ollama list.

O Ollama fica em loopback por padrão. keep_alive: 0 libera o modelo após o uso para poupar VRAM, com o custo de carregar novamente na próxima chamada. A primeira resposta pode demorar mais. VLADMIR_CONFIG seleciona outro arquivo; demais variáveis de configuração usam o prefixo VLADMIR_, como descrito em config/README.md. Não coloque tokens, senhas ou URLs privadas no Git.

Inicie a interface com ./scripts/vladmir-desktop no checkout ou vladmir-desktop após instalar o launcher. O comando ./scripts/vladmir abre o modo de voz/terminal, conforme opções da ferramenta.

5. Conversas, memória e modos

* Geral: chat principal com seletor de modelo e nível instantâneo, médio ou alto. O nível ajusta opções de geração/contexto no caminho de chat; não é uma garantia de que o modelo raciocina melhor.
* Engenheiro: área de programação com conversas próprias e ferramentas específicas. A disponibilidade real depende do modelo local e das ferramentas habilitadas.
* Professor: área de estudo com conversas e apoio de materiais da biblioteca.
* Memória coletiva: contexto e fatos podem ser compartilhados entre perfis/áreas pelo armazenamento local. A memória não equivale a reter toda a conversa para sempre. O tamanho do histórico enviado ao modelo é limitado pela configuração e pelo contexto do modelo.
* Honcho: integração opcional desativada por padrão. A configuração de exemplo mantém share_conversations falso. Ativar serviço remoto pode enviar dados; configure deliberadamente e revise privacidade antes.

O estado persistente fica em ~/.local/state/vladmir e ~/.local/share/vladmir. Faça cópia segura desses diretórios antes de migração ou limpeza. A localização exata pode variar por variável XDG ou configuração.

6. Anexar arquivos e usar a biblioteca

O chat do desktop aceita um conjunto de documentos de texto/estrutura, PDF, DOCX e imagens. Os limites atuais do chat são 5 anexos, até 6 MiB por arquivo, 8 MiB somados, e extração textual até 400.000 caracteres. A API móvel também impõe limites de corpo e anexos. Portanto, arquivo de 1 GB não é suportado no chat.

A Biblioteca é outro fluxo: importa e indexa referências para consulta posterior. Há extratores para documentos, planilhas, apresentações, ebooks, dados estruturados, texto e muitos formatos de código. Limites relevantes do extrator incluem 64 MiB de entrada, 32 MiB de conteúdo expandido de arquivo compactado e 2 milhões de caracteres extraídos. Arquivos não reconhecidos podem virar referência de metadados ou texto bruto; isso não significa compreensão semântica do formato.

Para importar: abra Biblioteca, selecione arquivos e acompanhe o estado da fila. Evite importar cópias enormes de diretórios, arquivos executáveis desconhecidos ou dados privados sem necessidade. PDFs digitalizados dependem de OCR instalado; áudio/vídeo depende de ferramentas locais e a extração é parcial. A biblioteca não treina automaticamente o modelo com os documentos.

7. Voz, chamada e áudio

O caminho de voz no PC captura áudio mono a 16 kHz, aplica detecção de fala e usa Whisper para transcrição. Para resposta falada, usa Piper e uma alternativa local quando disponível. O sistema escolhe a fonte e saída PulseAudio/PipeWire configuradas. A chamada no aplicativo processa turnos de áudio; não é um canal telefônico de latência garantida nem uma sessão de fala duplex de ponta a ponta.

Como usar:

1. Instale/configure o dispositivo de entrada e saída no sistema.
2. Configure os binários e modelos Whisper/Piper conforme config/config.example.json.
3. Use Chamada ou a ferramenta de voz e conceda acesso ao microfone quando o sistema pedir.
4. Confira a fonte e a saída no seletor de áudio do desktop se a música ou voz sair no dispositivo errado.
5. Encerre a chamada para parar a captura do microfone.

Diagnóstico inicial: pactl list short sources, pactl list short sinks, pactl get-default-source e pactl get-default-sink. Uma chamada Bluetooth pode fazer o sistema trocar o perfil do fone para HFP/HSP; esse perfil reduz a qualidade de reprodução. Escolher uma entrada separada e manter A2DP estéreo depende dos perfis e do servidor de áudio local.

8. Câmera e visão

No desktop, Vladmir pode capturar uma imagem de câmera USB ou um frame de fonte RTSP/RTSPS configurada e enviar a imagem ao modelo de visão local. A visão contínua limita frequência e usa mudanças de cena; ela não é gravação contínua nem análise de todos os quadros. Escolha explicitamente uma fonte, verifique permissões e teste a conexão antes de depender do recurso.

URLs RTSP podem conter usuário/senha: guarde-as somente em configuração local com permissão 0600. Uma checagem de URL não confirma a câmera física nem credenciais corretas. Feche a tela de visão para liberar a captura.

9. Web, GitHub e plugins

* Web: plugin nativo para pesquisa e leitura de páginas públicas. Requer internet, integração ativa e provedor disponível. O leitor limita destinos locais/privados para reduzir SSRF. Não é acesso irrestrito a qualquer rede.
* GitHub: pode ler repositórios públicos ou os acessíveis pela conta autenticada via gh. Edição é uma ação explícita no painel, com branch e confirmação. Não é um shell remoto genérico.
* Plugins: o projeto carrega integrações nativas revisadas (Web, GitHub, God’s Eye View). O painel informa permissões. Um marketplace para plugins arbitrários de terceiros não está implementado.
* JetBrains MCP: configure o servidor MCP HTTP local no PyCharm/CLion, cole a configuração/endereço em Ferramentas -> IDEs JetBrains, liste as ferramentas e confirme cada execução. Só aceite ferramentas e argumentos que entende. O servidor deve permanecer restrito a localhost.

Autenticação GitHub: instale e autentique gh conforme a documentação oficial antes de abrir o painel. Não cole token em conversa ou no repositório. O acesso efetivo depende das permissões da conta e do escopo do token.

10. Agenda, email, pacotes e Sentinel

* Agenda: itens e lembretes ficam em SQLite local. Pode criar, listar e concluir compromissos simples. Não é sincronizada com Google Calendar/Outlook.
* Email: a ferramenta prepara um mailto: com destinatário, assunto e corpo e abre o aplicativo configurado. O usuário revisa e envia; Vladmir não envia diretamente pelo Tuta.
* Instalação de pacotes: solicita instalação por gerenciadores permitidos, prepara inventário dos pacotes e exige confirmação na ação. O inventário é um registro do estado dos gerenciadores, não imagem de sistema nem restauração completa. Há restrições contra AUR/PKGBUILD e instalação irrestrita.
* Sentinel: interface para estado, varredura de caminho com ClamAV e investigação local.

Instale/atualize ClamAV e suas assinaturas por conta própria. A análise não cobre automaticamente todo o sistema com privilégios administrativos e um resultado limpo só se aplica aos itens que o ClamAV conseguiu examinar. Consulta VirusTotal por hash é opcional e desativada por padrão.

11. Autoevolução e treino

O mecanismo experimental reúne sinais técnicos, cria propostas e executa candidatos em ambiente isolado para avaliação. O candidato não é aplicado automaticamente ao programa de produção; promoção e rollback são ações protegidas. A rotina agendada depende de arquivos systemd do usuário preparados e ativação explícita.

O treino de adaptadores/QloRA é uma trilha separada. Exige contrato explícito de autorização, dataset revisado, hashes e limites de rodadas/tempo/VRAM. Resultado é um candidato e passa por comparação de qualidade/eficiência; não é prova de melhora geral e não troca o modelo automaticamente. Se os artefatos, contrato ou autorização não estiverem presentes, o treino não começa.

12. Android e Wear OS

O código nativo está em mobile-android/, vladmir-companion/ e vladmir-wear/. Para compilar, use Android Studio com SDK/JDK compatíveis e consulte os READMEs e Gradle wrappers. Instale um build de debug em aparelho de teste; builds publicados exigem assinatura configurada fora do Git.

O app móvel oferece chat com servidor configurado, envio de câmera, chamada por turnos de voz e fluxo ARCore opcional. A conectividade exige que o PC esteja acessível, API ligada, TLS/endereço adequados quando remoto, pareamento e permissões concedidas. AR depende de ARCore suportado e Google Play Services for AR. O código informa que o mapa AR só existe enquanto a tela está aberta; não há persistência automática do mapa.

O Companion encaminha notificações selecionadas e utilidades pelo Bluetooth pareado. WhatsApp é tratado como app selecionável; o preview textual tem limite curto e depende do acesso a notificações do Android. A tela de diagnóstico do Companion declara que reconexão automática ainda não está habilitada. Portanto, não conte com vínculo permanente em segundo plano sem validar no aparelho/versão atual.

Sensores listados dependem do hardware do relógio e permissões Android. A lista indica disponibilidade, não calibração clínica ou exatidão. Batimentos cardíacos, GPS, movimento, notificações e chamadas devem ser validados individualmente num aparelho real.

13. Privacidade, backups e segurança

* Mantenha ~/.config/vladmir/config.json em modo 0600 e não versionado.
* Faça backup de configuração, memória, agenda e biblioteca antes de atualizar.
* Mantenha a API móvel em loopback; para acesso remoto, use HTTPS/túnel autenticado e não exponha porta aberta à internet.
* Revise integrações externas ativadas, sobretudo Web, Honcho, GitHub e envio opcional de hash.
* Trate conteúdo de páginas, documentos e repositórios como dado não confiável.
* Consulte SECURITY.md para relato privado de vulnerabilidades e os arquivos de terceiros para licenças.

14. Checklist de diagnóstico

1. Rode ./scripts/vladmir --check e leia cada dependência ausente.
2. Confirme ollama list e teste se o serviço responde em 127.0.0.1:11434.
3. Veja o log do Vladmir sem copiar tokens, URLs privadas ou transcrições para suporte público.
4. Para áudio, confira fontes/saídas Pulse e perfil Bluetooth.
5. Para câmera, valide permissões e captura do frame separadamente do modelo.
6. Para celular/relógio, confirme API, endereço, TLS, pareamento, permissões e serviço em segundo plano.
7. Para biblioteca, confira formato real, tamanho, dependências de OCR/transcrição e estado do item.

8. Se falhar, registre versão/commit, passos mínimos, mensagem de erro sanitizada e se o defeito pode ser reproduzido.

15. Fontes internas

README raiz; config/config.example.json; src/mark/config.py; src/mark/desktop.py; src/mark/mobile_api.py; src/mark/attachment_limits.py; src/mark/cognition/library_extract.py; src/mark/audio.py; src/mark/tts.py; src/mark/vision.py; src/mark/plugins.py; src/mark/github.py; src/mark/jetbrains_mcp.py; src/mark/agenda.py; src/mark/package_install.py; src/mark/sentinel.py; src/mark/evolution/; mobile-android/README.md; vladmir-companion/; vladmir-wear/; SECURITY.md.